

**SIMATS ENGINEERING**  
**Department of Computer Science & Engineering**  
**ASSESSMENT TOOL 1-INDUSTRY PROBLEM SOLVING TASK**

---

|                       |                                                                                                                                                                                                                                                           |                      |                                                       |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|-------------------------------------------------------|
| <b>Course Code:</b>   | <b>CSA12</b>                                                                                                                                                                                                                                              | <b>Course Title:</b> | <b>Computer Architecture for Multicore Systems</b>    |
| <b>CO Assessed:</b>   | <b>CO4</b>                                                                                                                                                                                                                                                | <b>Assessment:</b>   | <b>ASSESSMENT TOOL1-INDUSTRY PROBLEM SOLVING TASK</b> |
| <b>Weightage:</b>     | <b>30%</b>                                                                                                                                                                                                                                                | <b>Total Marks:</b>  | <b>25</b>                                             |
| <b>CO4 Statement:</b> | <i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i> |                      |                                                       |
| <b>Student Name:</b>  | Ashika.B                                                                                                                                                                                                                                                  | <b>Reg.No.:</b>      | 192521336                                             |
| <b>Department:</b>    |                                                                                                                                                                                                                                                           | <b>Date:</b>         |                                                       |

**ANNEXURE A**  
**INDUSTRY PROBLEM SOLVING TASK**

1. Smartphone Performance Optimization

A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

2. Cloud Server Memory Bottleneck

A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

3. Autonomous Vehicle Data Processing

An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

4. AI Inference Edge Device

An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

5. High-Performance Gaming Console

A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

**Code of Conduct:**

I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

**MARKS ALLOCATION**

|    | Understanding of Industry Problem (20%) | Application of Engineering Concepts (25%) | Solution Design (25%) | Use of Tools (15%) | Analysis (10%) |
|----|-----------------------------------------|-------------------------------------------|-----------------------|--------------------|----------------|
| Q1 |                                         |                                           |                       |                    |                |
| Q2 |                                         |                                           |                       |                    |                |
| Q3 |                                         |                                           |                       |                    |                |
| Q4 |                                         |                                           |                       |                    |                |
| Q5 |                                         |                                           |                       |                    |                |

**RUBRICS FOR CALCULATION:**

| Criteria                            | Weightage | Level 4 – Exemplary                                                          | Level 3 – Proficient                                 | Level 2 – Developing                           | Level 1 – Beginning                               |
|-------------------------------------|-----------|------------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------------|---------------------------------------------------|
| Understanding of Industry Problem   | 20%       | Comprehensive understanding of real-world problem and constraints            | Good understanding with minor gaps in requirements   | Basic understanding; some requirements unclear | Poor understanding; misinterprets problem context |
| Application of Engineering Concepts | 25%       | Applies advanced and relevant concepts effectively to solve the problem      | Applies appropriate concepts with minor errors       | Limited application of concepts                | Incorrect or no application of concepts           |
| Solution Design                     | 25%       | Innovative, practical, and feasible solution aligned with industry standards | Logical and feasible solution with minor gaps        | Basic solution with limited feasibility        | Unclear or impractical solution                   |
| Use of Tools                        | 15%       | Effective use of modern tools, technologies, and real data                   | Appropriate use of tools with correct implementation | Limited or inconsistent use of tools           | Minimal or incorrect use of tools                 |

1. Smartphone Performance Optimization

Constraints

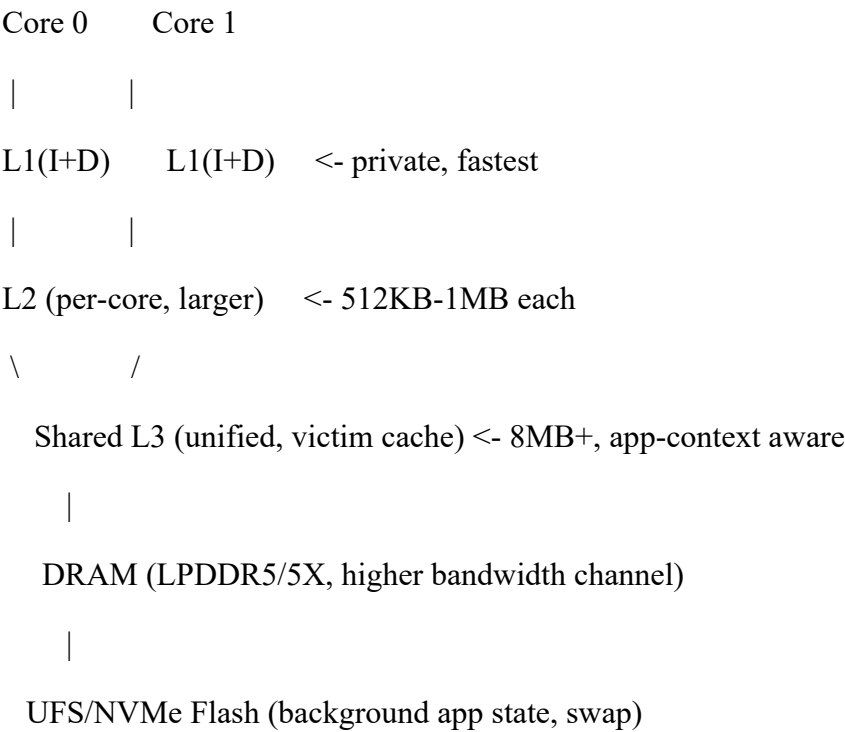
- Noticeable **lag while switching between apps**.
- Must analyze **L1, L2, L3 cache and DRAM** roles.
- Power budget is limited (mobile/battery-constrained device).
- Redesign must reduce **latency** and improve **throughput**.

Analysis of Memory Roles

**Why lag occurs during app switching:** when the user switches apps, the working set of the new app is not present in L1/L2/L3 — it must be fetched from DRAM, and if DRAM itself is under pressure (many background apps), pages may even be swapped from flash storage. This causes a **cache miss cascade** down the hierarchy, each miss adding latency.

Proposed Hierarchical Redesign

text



| Level    | Typical Size         | Latency   | Role                                                                      |
|----------|----------------------|-----------|---------------------------------------------------------------------------|
| L1 Cache | 32–64 KB per core    | ~1–2 ns   | Stores most frequently used instructions/data per core; fastest, smallest |
| L2 Cache | 256 KB–1 MB per core | ~4–10 ns  | Buffers between L1 and L3; holds recently evicted L1 data                 |
| L3 Cache | 2–8 MB, shared       | ~15–30 ns | Shared across cores; holds data relevant to                               |

|      |         |            |                                                                         |
|------|---------|------------|-------------------------------------------------------------------------|
|      |         |            | multiple app processes/threads                                          |
| DRAM | 4–12 GB | ~80–120 ns | Main memory holding all active app states, OS, and background processes |

### Working

1. **Increase L2 size per core** so each app's active working set fits closer to the core, reducing L3/DRAM trips during context switches.
2. **Make L3 a victim cache with app-aware prefetching** — when the OS scheduler signals an impending app switch (foreground activity change), the memory controller can prefetch the target app's recently used pages into L3 ahead of time.
3. **Use LPDDR5/LPDDR5X DRAM** instead of older LPDDR4X — higher bandwidth and lower access latency reduces the penalty of an L3 miss.
4. **Application context caching:** keep the last 2–3 recently used apps' critical working sets pinned in DRAM (not evicted), avoiding full reload from flash storage.
5. **Prefetching driven by OS scheduler hints** rather than pure hardware prediction, since app-switch is a known, predictable event on mobile OS.

### Justification

A larger, app-aware L3 acting as a victim cache reduces the frequency of DRAM accesses, which are the primary source of latency during app switches. Using LPDDR5/5X increases DRAM throughput and reduces the residual penalty when DRAM access is unavoidable. OS-assisted prefetching converts an unpredictable-looking latency spike into a hidden, pipelined operation, since app switching is a foreground/background OS-level event that can be anticipated.

### Conclusion

The redesign combines **larger per-core L2, an app-aware shared L3, LPDDR5/5X DRAM, and OS-assisted prefetching** to minimize the cache-miss cascade responsible for app-switch lag, improving both latency and throughput without materially increasing power draw.

## 2. Cloud Server Memory Bottleneck

### Constraints

| Aspect  | Cache Hierarchy (L1–L3)                  | Virtual Memory Paging             |
|---------|------------------------------------------|-----------------------------------|
| Purpose | Speeds up access to frequently used data | Extends addressable memory beyond |

| Aspect               | Cache Hierarchy (L1–L3)                     | Virtual Memory Paging                                  |
|----------------------|---------------------------------------------|--------------------------------------------------------|
|                      | within DRAM                                 | physical DRAM using disk/SSD                           |
| Latency              | Nanoseconds                                 | Microseconds to milliseconds (on page fault)           |
| Managed by           | Hardware (cache controller)                 | OS (page tables, TLB)                                  |
| Miss penalty         | Goes to next cache level or DRAM            | Goes to secondary storage (very expensive)             |
| Effect on DB queries | Reduces access time to hot index/data pages | A page fault stalls the query until disk I/O completes |

- High latency in **database queries**.
- Must compare **cache hierarchy vs virtual memory paging**.
- Recommend improvements using appropriate **memory technologies**.
- Server-scale workload (large, shared, concurrent).

### Cache Hierarchy vs Virtual Memory Paging

**Diagnosis:** Database queries typically access large working sets (indexes, buffer pools) that often exceed cache capacity, causing frequent **cache misses to DRAM**. When DRAM itself is oversubscribed (multiple tenants/VMs), the OS resorts to **paging to disk**, which is orders of magnitude slower and is the dominant contributor to high query latency — not the cache hierarchy itself.

### Proposed Improvements

text

CPU cores

|

L1/L2 (per-core)

|  
Shared L3 (larger, database-aware partitioning)

|  
DRAM (expanded capacity + NVDIMM tier)

|  
NVMe SSD (fast swap / storage tier, replacing paging to HDD)

### **Working**

1. **Increase DRAM capacity** so the database buffer pool (working set) fits in memory, minimizing page faults.
2. **Use cache partitioning (CAT – Cache Allocation Technology)** so noisy-neighbor VMs/tenants cannot evict the database process's hot cache lines.
3. **Introduce a memory tier using NVDIMM or persistent memory (Optane-class)** between DRAM and SSD, so that when paging is unavoidable, it is far faster than traditional disk swap.
4. **Replace HDD-based swap with NVMe SSD** for any residual paging, cutting page-fault penalty significantly.
5. **Tune the OS page replacement policy** to prioritize database buffer pool pages over less-critical process pages.

### **Justification**

Cache hierarchy optimizations reduce access latency for data that is already resident in DRAM, but they cannot fix the much larger penalty caused by page faults. Expanding DRAM and adding a fast NVDIMM/NVMe tier directly targets the actual bottleneck — the virtual memory paging penalty — while cache partitioning ensures multi-tenant workloads don't degrade each other's hit rates.

### **Conclusion**

The recommended fix is **DRAM capacity expansion + cache partitioning (CAT) + a persistent-memory/NVMe swap tier**, since the primary latency source is paging-to-storage, not the cache hierarchy.

## **3. Autonomous Vehicle Data Processing**

### **Constraints**

- Sensor data must be processed in **real time**.
- Must evaluate **SRAM, DRAM, and MRAM**.
- Design must **maximize bandwidth** and **minimize latency**.
- Safety-critical — data loss or excessive latency is unacceptable.

### **Technology Comparison**

## Proposed Memory Hierarchy

text

Sensors (LiDAR, camera, radar)

|

DMA ingestion

|

SRAM buffer (per-sensor, low-latency staging)

|

Shared SRAM/L2 cache (sensor fusion engine)

|

High-bandwidth DRAM (HBM/GDDR-class, model + fused data)

|

MRAM (non-volatile, critical state / black-box log)

## Working

1. Raw sensor streams (camera, LiDAR, radar) are DMA'd into **per-sensor SRAM buffers** for minimal-latency staging, since SRAM has near-zero access latency for time-critical fusion.
2. The **sensor fusion engine** (object detection, path planning) reads from a shared SRAM/cache layer, avoiding round trips to DRAM for time-critical intermediate results.
3. **High-bandwidth DRAM** (e.g., HBM or wide-bus LPDDR) holds the full fused world-model and inference model weights, since this data set is too large for SRAM but needs high throughput.
4. **MRAM** stores critical safety state (last known good trajectory, fault logs) because it is non-volatile — if power is interrupted, this state survives without needing a battery-backed write-back, unlike DRAM.
5. A **watchdog process periodically snapshots critical state to MRAM** without stalling the main real-time pipeline, since MRAM write latency is close to SRAM/DRAM.

## Justification

SRAM's near-instant access makes it ideal for the innermost real-time fusion loop where every nanosecond affects reaction time. DRAM (high-bandwidth variants) is necessary because sensor fusion and inference models require far more capacity than SRAM can economically provide. MRAM is included specifically for its non-volatility, which is a safety requirement — losing state on power interruption is unacceptable in an autonomous vehicle, and MRAM avoids the latency and energy cost of flash-based logging.

## Conclusion

The hierarchy of **SRAM (staging + fusion) → high-bandwidth DRAM (model/world-state) → MRAM (non-volatile safety state)** balances the vehicle's need for minimal latency, high throughput, and guaranteed retention of safety-critical data.

## 4. AI Inference Edge Device

### Constraints

- Needs **fast model loading**.
- Must operate under **low power consumption**.
- Must compare **NAND Flash, DRAM, and RRAM**.
- Recommend an optimized hierarchical structure.

### Technology Comparison

#### Proposed Memory Hierarchy

text

NAND Flash (full model repository, cold storage)

|

RRAM (hot model weights, non-volatile cache)

|

DRAM (active inference buffers, intermediate activations)

|

Inference engine (NPU/accelerator)

#### Technology Comparison

| Technology | Read Latency         | Write Latency    | Volatility   | Power                    | Best Use                             |
|------------|----------------------|------------------|--------------|--------------------------|--------------------------------------|
| NAND Flash | Slow ( $\mu$ s)      | Slow             | Non-volatile | Low standby              | Bulk model storage                   |
| DRAM       | Fast (ns)            | Fast             | Volatile     | Moderate (needs refresh) | Active inference working memory      |
| RRAM       | Fast (close to DRAM) | Fast, low-energy | Non-volatile | Very low                 | Fast, low-power model weight loading |

### Working

1. The **complete set of AI models** is stored in **NAND Flash**, since it is dense, cheap, and non-volatile — suitable for cold storage where speed is not critical.
2. On device wake, the **frequently used model (or its most-used layers)** is loaded into **RRAM** rather than DRAM directly, because RRAM read/write is much faster than NAND and consumes far less energy than repeatedly reloading from flash.
3. **DRAM** is used only for the active working memory during inference — intermediate activations, temporary buffers — since these require high-speed read/write but do not need persistence.



- 4. Because RRAM is non-volatile, the "hot" model weights **persist across power cycles**, so the device can resume inference almost instantly without re-reading from slow NAND flash, directly addressing the fast model-loading requirement.
- 5. Power is minimized because RRAM requires no refresh (unlike DRAM) and no high-voltage program/erase cycles (unlike NAND).

**Justification**

NAND flash alone is too slow for repeated model loading, and keeping the entire model resident in DRAM continuously wastes power due to refresh cycles. RRAM occupies the ideal middle ground — non-volatile like flash but fast and low-power like DRAM — making it the correct technology for a "hot cache" of frequently used model weights. This directly reduces cold-start latency while keeping idle power low.

**Conclusion**

The optimized hierarchy is **NAND Flash (cold model storage) → RRAM (fast, non-volatile hot-weight cache) → DRAM (inference working memory)**, achieving fast model loading with minimal power consumption.

**5. High-Performance Gaming Console**

**Constraints**

- **Frame drops** during large scene loading.
- Must analyze **L3 cache vs DRAM throughput**.
- Must propose memory hierarchy enhancements.
- Real-time rendering performance is critical.

**L3 Cache vs DRAM Throughput Analysis**

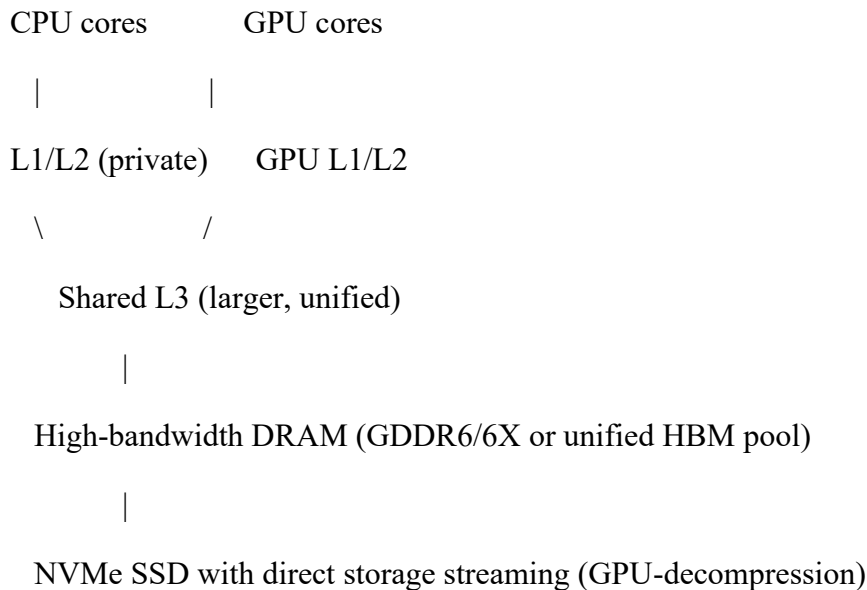
| Aspect                | L3 Cache                      | DRAM (e.g., GDDR6/DDR5)                             |
|-----------------------|-------------------------------|-----------------------------------------------------|
| Latency               | Very low (~10–20 ns)          | Higher (~80–100 ns)                                 |
| Bandwidth             | High but limited by size      | High sustained bandwidth, but shared across CPU+GPU |
| Capacity              | Small (a few MB)              | Large (8–16 GB)                                     |
| Role in scene loading | Cannot hold full scene assets | Must stream large textures, geometry, and           |

world data

**Diagnosis:** Large scene loading requires streaming large volumes of texture and geometry data that far exceed L3 cache capacity — this data must come from DRAM, and ultimately from storage (SSD). Frame drops occur when the **DRAM bandwidth is saturated** by simultaneous CPU (game logic) and GPU (rendering) demands during a load, or when data isn't prefetched in time and the GPU stalls waiting on memory.

### Proposed Memory Hierarchy Enhancement

text



### Working

1. **Enlarge and partition L3 cache** between CPU and GPU workloads so critical, small, reused data (game logic state, shader constants) isn't evicted by bulk streaming traffic.
2. **Use high-bandwidth DRAM (GDDR6/6X or unified memory pool)** shared between CPU and GPU, sized and clocked to sustain simultaneous game-logic and rendering bandwidth demands.
3. **Implement direct storage streaming (SSD → GPU-decompressed → DRAM)**, bypassing traditional CPU-mediated file I/O, similar to modern console "Velocity Architecture" style designs — this reduces the load-time bottleneck at its source.
4. **Predictive asset prefetching:** as the player approaches a new area, stream upcoming scene assets into DRAM ahead of time rather than reactively on demand.
5. **Compression-aware pipelines:** decompress assets close to the GPU to reduce the effective bandwidth needed to move data from SSD to DRAM.

### Justification

Because L3 cache cannot hold the volume of data required for large scene loads, DRAM throughput and the path from storage to DRAM are the actual bottlenecks. Direct storage streaming and predictive prefetching remove the CPU as a serialization point and hide load latency behind

gameplay, while a larger, partitioned L3 protects latency-sensitive game logic from being disrupted by bulk streaming traffic.

## Conclusion

Frame drops during scene loading are primarily a **DRAM/storage bandwidth problem**, not an L3 cache-size problem. The fix combines **partitioned L3 cache, high-bandwidth unified DRAM, direct SSD-to-GPU streaming, and predictive prefetching** to sustain frame rate through large asset loads.

## Quick Final Summary

| Question | Best Design                                                                                  |
|----------|----------------------------------------------------------------------------------------------|
| 1        | Larger per-core L2 + app-aware shared L3 + LPDDR5/5X DRAM + OS-assisted prefetch             |
| 2        | Expanded DRAM + cache partitioning (CAT) + NVDIMM/NVMe swap tier                             |
| 3        | SRAM (fusion staging) → high-bandwidth DRAM (world model) → MRAM (non-volatile safety state) |
| 4        | NAND Flash (cold storage) → RRAM (hot weight cache) → DRAM (inference buffer)                |
| 5        | Partitioned L3 + high-bandwidth unified DRAM + direct SSD-to-GPU streaming + prefetch        |